



Plan de gestión, análisis y diseño y memoria del proyecto Snow Party

15/03/2026

Grupo 05 - Carol Shaw

Lorién Blasco (901218)
Aritz Casado (901675)
Alejandro Sáenz (900280)
Daniel Hidalgo (898915)
Juan Manuel Fano (901231)
Eduardo Blasco (903723)
Alonso de la Calle (896554)
David Salas (899166)

Organización de Github:
<https://github.com/UNIZAR-30226-2026-05>

Índice

[1. Introducción](#)

[2. Organización del proyecto](#)

[3. Plan de gestión de proyecto](#)

[3.1. Inicio del Proyecto](#)

[3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto](#)

[3.3. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento](#)

[4. Análisis y diseño del sistema](#)

[4.1 Análisis de requisitos](#)

[Minijuegos de Orden \(Todos a la vez\)](#)

[Minijuegos de Casilla \(Durante el turno\)](#)

[4.2 Diseño del sistema](#)

[5. Memoria del proyecto](#)

[5.1 Inicio del proyecto](#)

[5.2 Ejecución y control del proyecto](#)

[5.3 Cierre del proyecto](#)

[Anexo I - Glosario](#)

1. Introducción

Este documento define el plan de gestión, análisis y diseño para el desarrollo de Snow Party, un juego multijugador que combina la mecánica clásica del juego de la oca con minijuegos de habilidad y un sistema de economía interna. El propósito del proyecto es construir un sistema completo y escalable, desde el diseño de la base de datos y la lógica del servidor, hasta el desarrollo de dos clientes independientes: una aplicación nativa de escritorio y una interfaz web.

El objetivo técnico principal es garantizar una experiencia fluida y sincronizada en tiempo real para 4 jugadores simultáneos. Para ello, el sistema debe gestionar de forma eficiente la lógica de turnos, la concurrencia durante la resolución de los minijuegos y la persistencia de datos de los usuarios, permitiendo la continuidad de la sesión entre diferentes dispositivos sin pérdida de información.

El desarrollo se estructura en torno a dos fechas. La primera fecha corresponde a la Entrega Intermedia (15 de marzo de 2026), donde se presenta un Producto Mínimo Viable (MVP) con el núcleo del juego funcional (con demostración en video para el 6 de abril). La segunda fecha es la Entrega Final (4 de mayo de 2026), que incluirá todas las mecánicas avanzadas y minijuegos. Al terminar el proyecto, los entregables serán: el código fuente alojado en repositorios, el software completamente desplegado y accesible, los instaladores para la versión de escritorio, y la documentación técnica y manuales correspondientes.

Para detallar el ciclo de vida del proyecto, el resto de este documento se organiza de la siguiente manera:

El apartado 2 expone la organización del equipo y la asignación de roles.

El apartado 3 detalla el plan de gestión, el control de configuraciones, el calendario (diagrama de Gantt) y las herramientas de coordinación.

El apartado 4 profundiza en el análisis de requisitos funcionales y no funcionales, así como en el diseño de la arquitectura y la justificación de las tecnologías elegidas, además del diseño gráfico del sistema.

Finalmente, el Capítulo 5 recoge la memoria del proyecto, evaluando la realidad frente a lo planificado y analizando el esfuerzo invertido hasta la fecha actual.

2. Organización del proyecto

El proyecto será desarrollado por un equipo de 8 ingenieros de software, organizados en sub-equipos especializados para cubrir todas las áreas del desarrollo *Full Stack*. El equipo cuenta con experiencia en el desarrollo de aplicaciones web modernas, sistemas distribuidos y desarrollo multiplataforma.

Miembro	Rol Principal	Responsabilidades Clave
Lorién Blasco*	Coordinador & Diseño / Front	Gestión del proyecto, diseño UI/UX y desarrollo del cliente Escritorio.
Alonso de la Calle	Configuración & Backend	Gestión de configuración (GitHub), arquitectura de la API y lógica del servidor.
Aritz Casado	Backend & Base de Datos	Diseño e implementación de la BDD (PostgreSQL) y lógica de negocio.
Eduardo Blasco	Backend & DevOps	Desarrollo del Backend y responsable del despliegue (Docker).
David Salas	Frontend Web	Desarrollo de la interfaz y lógica del cliente Web (React.js).
Juan Manuel Fano	Frontend Web	Desarrollo de la interfaz y lógica del cliente Web (React.js).
Alejandro Sáenz	Frontend Escritorio	Desarrollo de la aplicación de escritorio multiplataforma (Flutter).
Daniel Hidalgo	Diseño & Front Escritorio	Diseño de activos gráficos y desarrollo en Flutter.

**Lorién Blasco actuará como director del proyecto*

El equipo cuenta con un perfil técnico y experiencia demostrada en las tecnologías seleccionadas para este proyecto, lo que garantiza una ejecución eficiente y de calidad.

- **Desarrollo Web:** Experiencia en React.js para la creación de interfaces dinámicas y reactivas.
- **Desarrollo Multiplataforma:** Competencia en Flutter para el desarrollo nativo de escritorio.
- **Backend y API:** Dominio de Python (FastAPI) y/o Node.js para la creación de servicios RESTful eficientes y escalables.
- **Base de Datos:** Conocimientos avanzados en diseño y administración de bases de datos relacionales, específicamente PostgreSQL.
- **DevOps e Infraestructura:** Capacidad para la contenerización de aplicaciones mediante Docker y gestión de integración continua.

3. Plan de gestión de proyecto

3.1. Inicio del Proyecto

Identificación y asignación de recursos

Al realizar el proyecto en web y escritorio, descartamos la necesidad de dispositivos móviles para pruebas. Necesitaremos alquilar un servidor (Virtual Private Server / PaaS) en el cual alojaremos la base de datos, el Backend el cual se encargará de gestionar toda la lógica del juego, así como las partidas y la interfaz web. En este momento barajamos varias opciones como pueden ser: Hetzner (Europa) o DigitalOcean, Railway, Render o Fly.io siendo las dos primeras opciones las que más tendremos en cuenta a la hora de elegir.

Gestión de riesgos principales

Durante la fase inicial, hemos identificado varios riesgos críticos: la falta de experiencia previa trabajando juntos en este formato (Ri2) , enfrentarnos a funcionalidades que no están completamente definidas desde el principio (Ri6) , la complejidad técnica de construir un sistema de esta envergadura (Ri7) y la elección de tecnologías que el equipo conoce poco o nada (Ri8). Nuestra estrategia principal para mitigar todos estos riesgos se basa en centralizar la coordinación a través de la figura del responsable del proyecto (Lorién) y en establecer comunicaciones claras mediante reuniones periódicas, tanto a nivel general como en los diferentes grupos de trabajo. Como los grupos de trabajo son pequeños (2 o 3 personas) creemos que se gestionará correctamente.

Se puede consultar el plan de gestión de riesgos y sus estrategias de mitigación completos en este documento: [📄 Gestion_Riesgo_05_Carol_Shaw](#)

Plan de formación

La necesidad de formación se da sobre todo en los miembros encargados del frontend (tanto web como escritorio) ya que en el caso del backend ya se conocían las tecnologías. Para el aprendizaje de flutter, dart, nextJs y React se adoptará un enfoque autodidacta con los recursos disponibles en internet. Se intentará desarrollar ejemplos prácticos muy sencillos y similares a las funcionalidades del proyecto antes de implementarlas en el código del proyecto real.

3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto

Responsables

Como se ha mencionado en el apartado 2, Eduardo Blasco y Alonso de la Calle son los responsables técnicos de esta sección.

Gestión de repositorios y control de versiones

El código se estructurará utilizando cuatro repositorios independientes: uno para el backend, uno para el frontend web, uno para el frontend de escritorio y un cuarto repositorio destinado a la gestión y organización general del proyecto.

La documentación de gestión y organización del equipo, incluyendo el enlace al Excel de control temporal y reparto de tareas, se almacena de forma centralizada en el repositorio de organización para que sea accesible y editable por todos los miembros.

El flujo de trabajo es el siguiente: todos los repositorios tienen una única rama principal (main) sobre la que los miembros realizan commits directamente, sin requerir pull requests ni revisión previa. Se ha decidido así por el tamaño reducido del equipo y la comunicación constante entre los miembros, que evita conflictos y simplifica todo. Los issues de GitHub se utilizan para registrar tareas y errores. La decisión de cerrar un issue es de quien lo tiene asignado, notificándolo al resto del equipo por Discord.

Construcción, dependencias e integración

La construcción del backend se realiza mediante Docker Compose, que gestiona en contenedores tanto el servidor FastAPI como la base de datos PostgreSQL. Cada desarrollador levanta el entorno en local de forma idéntica a partir del fichero docker-compose.yaml incluido en el repositorio, garantizando consistencia. La gestión de dependencias se realiza mediante pip y requirements.txt en Python, pub y pubspec.yaml en Dart, y npm y package.json en TypeScript. La integración se realiza manualmente, comprobando que el sistema funciona.

Despliegue e infraestructura

Desarrollaremos nuestra aplicación para que pueda ser usada en el navegador Google Chrome Versión 144.0.7559.133 (build oficial) (64 bits), especialmente enfocaremos el diseño de la web para que pueda ser jugado en un ordenador, así como en Windows 11 21H2 (build 22000) o superior para la aplicación nativa de escritorio.

Para poder jugar al juego en cualquier momento del día, Si tenemos en cuenta el despliegue de nuestra aplicación (véase *Figura 3, apartado 4.2*), observamos cómo vamos a desplegar la aplicación en 3 tiers:

Tier 1: Base de datos, **Tier 2:** Backend + API Rest, **Tier 3:** Cliente (Web + Aplicación).

A la hora de agrupar estos, almacenaremos la base de datos, el Backend y el cliente Web en un servidor (comentado anteriormente) y el cliente de la aplicación en el dispositivo correspondiente.

Se explica más adelante en detalle en el apartado 4.2.

Entrega de resultados al cliente

Los resultados se entregarán al cliente mediante el código fuente accesible en los repositorios de la organización de GitHub (<https://github.com/UNIZAR-30226-2026-05>), el software desplegado en servidor (se proporcionará una URL), el instalador de la aplicación de escritorio, y la documentación técnica en formato PDF entregada a través de moodle.

Convenciones

Se ha decidido comentar el código de manera exhaustiva de forma que, aunque una persona no haya realizado esa parte, pudiera modificar en caso de cambio de roles o reorganización del equipo. En el caso de la organización de carpetas en el front-end, hemos tratado de hacerlas lo más homogéneas posible.

Aseguramiento de la calidad

Dado el fuerte componente visual, interactivo y multijugador del proyecto, la estrategia de validación se centrará en pruebas manuales estructuradas, priorizando la experiencia de usuario y la sincronización en tiempo real. Para garantizar la estabilidad, aplicaremos las siguientes metodologías:

- **Casos de prueba con Fast Api:** Utilizaremos las pruebas existentes para cada endpoint en la interfaz de Fast API, que nos permiten ejecutar las consultas individualmente tanto para los post (que permite introducir json de entrada) como los get (nos devuelve los resultados correspondientes) y recibiendo el código de resultado.
- **Casos de prueba concretos:** Definiremos un listado de escenarios críticos (ej. desconexiones de usuarios en mitad de un turno, colisiones en casillas, compras en la tienda) y los probaremos en el juego con modificaciones del código para forzar estos escenarios.
- **Pruebas cruzadas (Cross-testing):** Para evitar sesgos de desarrollo, fomentaremos que los desarrolladores del cliente Web validen el cliente de Escritorio y viceversa, reportando cualquier anomalía visual o de rendimiento.
- **Sesiones de Playtesting:** Al final del proyecto, se probarán partidas completas para detectar anomalías en el juego y que los requisitos se cumplan por completo.

3.3. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento

Calendario del proyecto

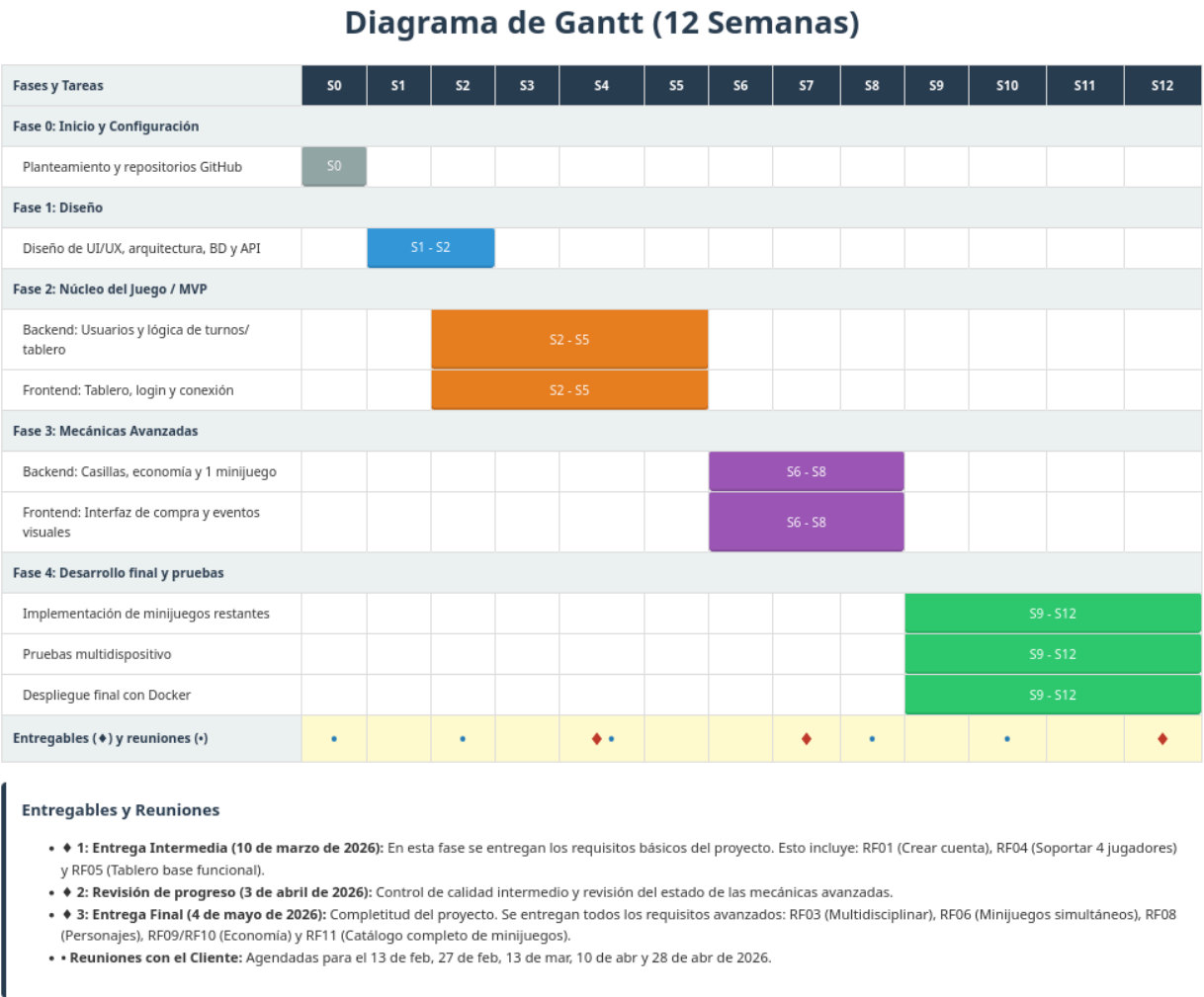


Figura 1: Diagrama de Gantt

Se detalla el plan de trabajo: (explicación de Figura 1: Diagrama de Gantt)

Fase 0: Inicio y Configuración (Semana 0)

- Planteamiento del proyecto.
- Configuración del entorno de desarrollo y repositorios (GitHub).

Fase 1: Inicio y Configuración (Semanas 1-2)

- Definición de requisitos detallados y diseño de la arquitectura.
- Diseño de la interfaz de usuario (UI/UX).
- Definición de la base de datos y API.

Fase 2: Núcleo del Juego (Semanas 2-5)

- **Objetivo:** Lograr un MVP (Producto Mínimo Viable) funcional.
- **Desarrollo del Backend:** Gestión de usuarios, autenticación y lógica básica de la partida (tablero y turnos).
- **Desarrollo Frontend:** Implementación del tablero, registro/login y conexión con el Backend.
- **Entrega Intermedia (Fecha estimada: 10 de marzo 2026)**
 - *Entregable:* Documentación de análisis y diseño preliminar.
 - *Entregable:* Versión funcional del juego con mecánica básica (tirar dados, mover fichas) accesible vía Web y escritorio.

Fase 3: Mecánicas avanzadas (Semanas 6-8)

- **Objetivo:** Implementar las reglas específicas del modo "Isla Aventura" (economía, objetos), las casillas especiales, asegurando la sincronización entre clientes Web y Escritorio.
- **Desarrollo del Backend:**
 - Implementación de lógica de casillas especiales (retrocesos, penalizaciones, eventos).
 - Sistema de economía: Lógica para ganar/perder monedas y comprar ventajas (tienda).
 - Implementación de un minijuego.
- **Desarrollo Frontend (Web y Escritorio):**
 - Interfaz de la compra.
 - Adaptación visual de los eventos del tablero.
- **Revisión de Progreso / Entrega Intermedia (Fecha estimada: 3 de abril 2026)**
 - *Entregable:* Primera versión funcional con las principales características implementadas.
 -

Fase 4: Desarrollo final y pruebas (Semanas 9-12)

- **Objetivo:** Implementación completa de minijuegos y prueba de despliegue y posibilidad de continuar la partida en otro dispositivo.
- **Desarrollo de Minijuegos:** Implementación de la lógica de los minijuegos (apuestas, reflejos, azar) para determinar el orden de turnos.
- **Despliegue:** Configuración final de los contenedores Docker y despliegue en servidor.
- **Entrega Final (Fecha estimada: 4 mayo 2026)**
 - *Entregable:* Memoria final del proyecto (Gestión, Análisis, Diseño).
 - *Entregable:* Manuales de despliegue.
 - *Entregable:* Código fuente completo en GitHub.
 - *Entregable:* Software desplegado y accesible para su evaluación.

Coordinación, reparto de tareas y comunicaciones

El reparto de tareas a bajo nivel se gestiona mediante Issues en nuestros repositorios de GitHub. Durante las reuniones asignamos estas tareas a los miembros del equipo en función de su rol.

Para la comunicación interna utilizamos:

- WhatsApp: Como canal general para avisos rápidos y coordinación diaria.
- Discord: Para el trabajo técnico, estructurado en canales específicos (frontend web, frontend escritorio, backend, diseño, etc.) que permiten organizar las discusiones de cada sub-equipo.
- Excel para control horario: Todos los miembros apuntan sus horas, tareas y resumen para poder saber qué hacen los demás miembros sin necesidad de preguntar.

Las decisiones clave de cada reunión se documentan ya sea en forma de actas para las primeras reuniones en las que se toman muchas decisiones o cómo issues dentro de github para mantener un registro.

Responsable

El responsable de esta sección será el director del proyecto: Lorién Blasco. En caso de bloqueo o disputa técnica, la decisión final recaerá sobre el director del proyecto tras escuchar a las partes.

Monitorización y seguimiento del progreso

El progreso del proyecto se monitoriza combinando dos métricas principales. Por un lado, revisamos los Issues cerrados en GitHub y las funcionalidades ya implementadas. Por otro lado, utilizamos un documento Excel organizativo compartido donde cada miembro registra las horas dedicadas a sus tareas. Esto nos permite tener un control temporal exacto y comparar el esfuerzo real que estamos invirtiendo con las estimaciones de costes y horas que hicimos al inicio del proyecto. Además, esta segunda métrica nos permite ver que, aunque el proyecto no avance tan rápido al principio, con el paso del tiempo se requiere menor tiempo para obtener más resultados.

4. Análisis y diseño del sistema

4.1 Análisis de requisitos

Se ha decidido separar los requisitos entre requisitos del sistema y requisitos por cada minijuego:

Requisitos funcionales del sistema:

- **RF01** - El sistema debe permitir a los usuarios crear una cuenta y acceder a ella.
- **RF02** - Los usuarios podrán seguir a otros jugadores y ver quiénes son sus seguidores para permitir la creación de partidas conjuntas.
- **RF03** - El sistema debe permitir la continuidad de la sesión y multidispositivo.
 - **RF03.1** - El usuario puede cambiar de dispositivo (web/escritorio) en cualquier momento de la partida.
 - **RF03.2** - El usuario puede iniciar su sesión en un segundo dispositivo, y el sistema cerrará su sesión en el primero automáticamente.
- **RF04** - El juego debe soportar 4 jugadores conectados simultáneamente.
- **RF05** - El juego consta de un tablero de 70 casillas de largo. Las casillas pueden tener minijuegos, u otras consecuencias (avanzar/retroceder x casillas, bloqueos, intercambio con otros jugadores, obtener premios, etc.).
- **RF06** - Antes de cada ronda de movimiento, todos los jugadores participarán en un Minijuego simultáneo. El resultado del minijuego determinará el orden de tirada y la calidad de los dados asignados (oro/plata/bronce).
 - **RF06.1** - Los dados se asignan según el ranquin del minijuego de la siguiente manera:
 - 1ª Posición: 1 dado normal (1-6) + 1 dado oro (1-6).
 - 2ª Posición: 1 dado normal + 1 dado plata (1-4).
 - 3ª Posición: 1 dado normal + 1 dado bronce (1-2).
 - 4ª Posición: 1 dado normal único.
- **RF07** - Si un jugador no interactúa durante un tiempo predefinido, el sistema salta su turno automáticamente. (El juego no se pausa ni juega por él).
- **RF08** - Al inicio del juego, cada jugador elegirá un personaje distinto con habilidades únicas de entre los siguientes:
 - Banquero: Roba una cantidad de monedas X a un jugador elegido por el propio banquero en cada turno.
 - Videojugador: Vota entre dos opciones de minijuegos posibles; se jugará el más votado por los personajes de esta clase.

- Escapista: Recibe penalizaciones reducidas en eventos negativos (ej. perder menos monedas, retroceder menos casillas o esperar menos turnos en las casillas de penalización de tiradas).
- Vidente: Puede visualizar el resultado de los dados antes de jugar el minijuego de orden para tomar decisiones estratégicas.
- **RF09** - Existe una moneda interna que se reinicia en cada partida. Cada turno los jugadores reciben una cantidad igual por turno.
- **RF10** - Los jugadores pueden canjear monedas por objetos estratégicos entre los siguientes.
 - Avanzar/Retroceder casillas.
 - Mejorar/Empeorar dados.
 - Barreras de bloqueo temporal.
 - Robar monedas a otro jugador.
 - Ruleta (premio al azar).
 - Quitar turno a un rival.
 - Salvavidas (eliminar penalización).
- **RF11** - El sistema implementa la lógica y control de los siguientes minijuegos, asegurando la sincronización entre todos los jugadores.
 - Minijuegos de Orden (Todos a la vez):
 - Tren: Contar pasajeros que pasan rápidamente.
 - Reflejos: Reacción ante cambio de color (Rojo a Verde).
 - Cortar pan: Precisión para cortar una barra en dos trozos iguales.
 - Cronómetro ciego: Detener un contador oculto en el momento exacto.
 - Mayor o Menor: Elección de cartas por suerte (carta alta gana).
 - Minijuegos de Casilla (Durante el turno):
 - Dilema del Prisionero (1 vs 1), cuando caen 2 jugadores en la misma casilla: Mecánica de cooperar/traicionar con recompensas variables (0, 2 o 3 monedas).
 - Doble o Nada: Apuesta de monedas contra la suerte (cara/cruz).
 - Mano de Póker: Versión simplificada con apuestas.

Requisitos no funcionales:

- **RNF1:** Los clientes se comunicarán mediante protocolo HTTPS con el Backend.
- **RNF2:** Se podrá jugar al juego en el navegador Google Chrome Versión 144.0.7559.133 (Build oficial) (64 bits).
- **RNF3:** Se podrá jugar al juego en ordenadores con sistema operativo Windows 11 21H2 (build 22000) o superior.

*Requisitos funcionales de los minijuegos:***Minijuegos de Orden (Todos a la vez)**

- **Tren**
 - **RF-M01:** El sistema mostrará una animación de un tren cruzando la pantalla con una cantidad aleatoria (entre X e Y) de pasajeros visibles.
 - **RF-M02:** El jugador que introduzca la cifra con la menor diferencia absoluta respecto al número real obtendrá la primera posición. En caso de empate, el sistema lanzará un dado interno para desempatar.
- **Reflejos**
 - **RF-M03:** El sistema mostrará una pantalla roja que cambiará a verde tras un intervalo de tiempo aleatorio (entre 2 y 5 segundos).
 - **RF-M04:** El sistema registrará el tiempo de reacción en milisegundos desde el cambio de color hasta la pulsación de la tecla. El menor tiempo determina el ganador.
- **Cortar pan**
 - **RF-M05:** La interfaz mostrará un indicador moviéndose horizontalmente sobre un objeto a velocidad constante. El jugador debe detenerlo pulsando una tecla.
 - **RF-M06:** Se calculará el porcentaje de desviación respecto al centro exacto (50%). El jugador con el porcentaje más cercano al 50% gana.
- **Cronómetro ciego**
 - **RF-M07:** Se mostrará un cronómetro en pantalla que se ocultará tras los primeros 3 segundos. El jugador debe detenerlo intentando clavar una marca temporal objetivo (ej. 10.00 segundos).
 - **RF-M08:** El sistema ordenará a los jugadores según la diferencia absoluta de tiempo entre su tiempo de parada y el tiempo objetivo.
- **Mayor o Menor**
 - **RF-M09:** El sistema presentará 4 cartas boca abajo. La selección de carta por parte de los jugadores se procesará en orden del turno anterior.
 - **RF-M10:** Al revelar las cartas, el sistema asignará las posiciones basándose en el valor numérico de la carta elegida (la más alta gana).

Minijuegos de Casilla (Durante el turno)

- **Dilema del Prisionero**

- **RF-M11:** Se activará una instancia 1vs1 estricta y secreta cuando dos jugadores coincidan en la misma casilla. Tendrán 10 segundos para elegir "Callar" o "Delatar".
- **RF-M12:** El Backend aplicará estos pagos: Ambos callan (+2 monedas c/u), uno delata (+3 el delator, 0 el otro), ambos delatan (0 monedas).

- **Doble o Nada**

- **RF-M13:** El jugador debe introducir una apuesta válida (mayor a 0 y menor o igual a su saldo actual).
- **RF-M14:** El sistema calculará un 50% de probabilidad real. Si es victoria, sumará la cantidad apostada al saldo; si es derrota, la restará.

- **Mano de Póker**

- **RF-M15:** El sistema validará que los participantes posean el saldo mínimo para la apuesta obligatoria inicial antes de repartir cartas. **NO SE PERMITE APOSTAR MÁS DINERO DEL QUE SE TIENE.** En caso de que un jugador no pueda igualar otra apuesta, no puede jugar y se le devuelve su dinero. *(De esta forma, el jugador con más dinero no puede dejar fuera a todos y llevarse la apuesta mínima de los demás, tiene que adaptarse al dinero que tienen los demás).*
- **RF-M16:** Se jugará con las reglas de poker una mano reducida con 2 cartas diferentes para cada jugador además de 3 comunes en el centro de la mesa.
 - **RF-M16.1:** En orden de turno, se empieza apostando con las reglas del poker, luego se muestran las 3 cartas centrales y se vuelve a apostar.
 - **RF-M16.2:** El ganador se elige según las reglas de poker entre los que quedan apostando en el momento final del juego.

4.2 Diseño del sistema

Arquitectura y Protocolos de Comunicación

El sistema se ha diseñado siguiendo un estilo arquitectónico Cliente-Servidor distribuido en tres capas (Tier 1: Base de Datos, Tier 2: Backend, Tier 3: Clientes Web y Escritorio) . Para cumplir con los objetivos funcionales del proyecto (gestión de datos persistentes e interacción en tiempo real), hemos optado por dividir la capa del servidor en dos protocolos de comunicación distintos:

- **API RESTful (HTTP/HTTPS):** Implementada con Fast API (Python), se encarga exclusivamente de las operaciones transaccionales y de estado persistente.
 - **Uso:** Registro y autenticación de usuarios, gestión del sistema de amigos (seguidores), consulta del inventario de la tienda y configuración inicial de la sala de juego.
- **Servidor de WebSockets (WS/WSS):** Integrado en el mismo entorno de backend, mantiene conexiones bidireccionales, persistentes y de baja latencia con los clientes.
 - **Uso:** Sincronización estricta del estado de la partida, broadcast de los lanzamientos de dados, posicionamiento en el tablero, control de turnos (y saltos por inactividad) y la ejecución simultánea de los minijuegos.

Justificación de las tecnologías elegidas para Backend:

Doble Protocolo (API REST + WebSockets):

Se ha barajado esta opción para almacenar los datos no variables en la base de datos, como los personajes del juego, los objetos o el tablero y los websockets para el almacenamiento de datos en tiempo de partida, como el saldo de cada jugador o la casilla en la que se encuentra.

Websockets:

No se barajaron más alternativas a los websockets. Era la mejor tecnología para la conectividad cliente-servidor en tiempo real, muy útil especialmente para los minijuegos donde ocurren una gran cantidad de cambios en poco tiempo donde hacer consultas a la base de datos provocaría mucha latencia.

Backend con Python (FastAPI):

Como opciones para el desarrollo de nuestra REST API barajamos Spring Boot y FastAPI. Con la primera trabajaríamos en java mientras que con la segunda trabajaríamos en Python.

Hemos escogido trabajar con FastAPI por las siguientes razones:

- Entorno de trabajo en Python, mucho más rápido y sencillo que trabajar programando en java
- Experiencia previa en el uso de FastAPI
- Extensa documentación generada automáticamente al utilizar FastAPI la cual hace mucho más sencillo el trabajo de los equipos de frontend. Tan solo con poner "/docs" al final del endpoint de la API conseguimos esta extensa documentación.

Base de Datos Relacional (PostgreSQL):

Se ha elegido para el diseño de la base de datos el sistema gestor postgresSQL ya que es una tecnología que ya conocíamos, fácil de implementar, ligera y gratuita lo que nos permite hacer el número de pruebas que necesitemos sin necesidad de preocuparnos por licencias.

Justificación de las tecnologías elegidas para Frontend:**Flutter - Frontend escritorio:**

Inicialmente se valoró Unity por la ligera experiencia previa del equipo. Sin embargo, se descartó por tener una curva de aprendizaje alta. Optamos por Flutter por rendimiento y rapidez para crear interfaces dinámicas. La utilización de Flame, un motor 2D ligero para Flutter que nos aporta exactamente lo necesario (gestión de sprites, animaciones y lógica) fue otro de los puntos a favor.

NextJs y React - Frontend web:

Para el desarrollo del frontend web buscamos información sobre 3 frameworks los cuales tienen una muy buena reputación en el mercado actual de desarrollo web. Estos tres eran Next.js + React, Vue y Angular.

- Angular es un buen framework, bastante estandarizado en la industria por su robustez y mantenido por google.
- Vue es un entorno ligero y por su reactividad es sencillo de aprender y rápido de implementar.
- Next.js es de entre los tres el framework que mejor funciona con la librería de React, la cual es la librería más famosa de componentes reutilizables. Debido a la popularidad de React en proyectos de desarrollo web, contamos con una gran cantidad de recursos on-line para aprender esta tecnología. Además, Next cuenta con Vercel el cual ofrece un servicio gratuito para desplegar el frontend lo cual valoramos muy positivamente.

Es por esto que decidimos trabajar con Next.js y React para desarrollar nuestro frontend web.

Diagramas arquitecturales

Para ilustrar mejor el sistema, se muestran a continuación los diagramas de módulos (Figura 2), despliegue (Figura 3) y componentes (Figura 4).

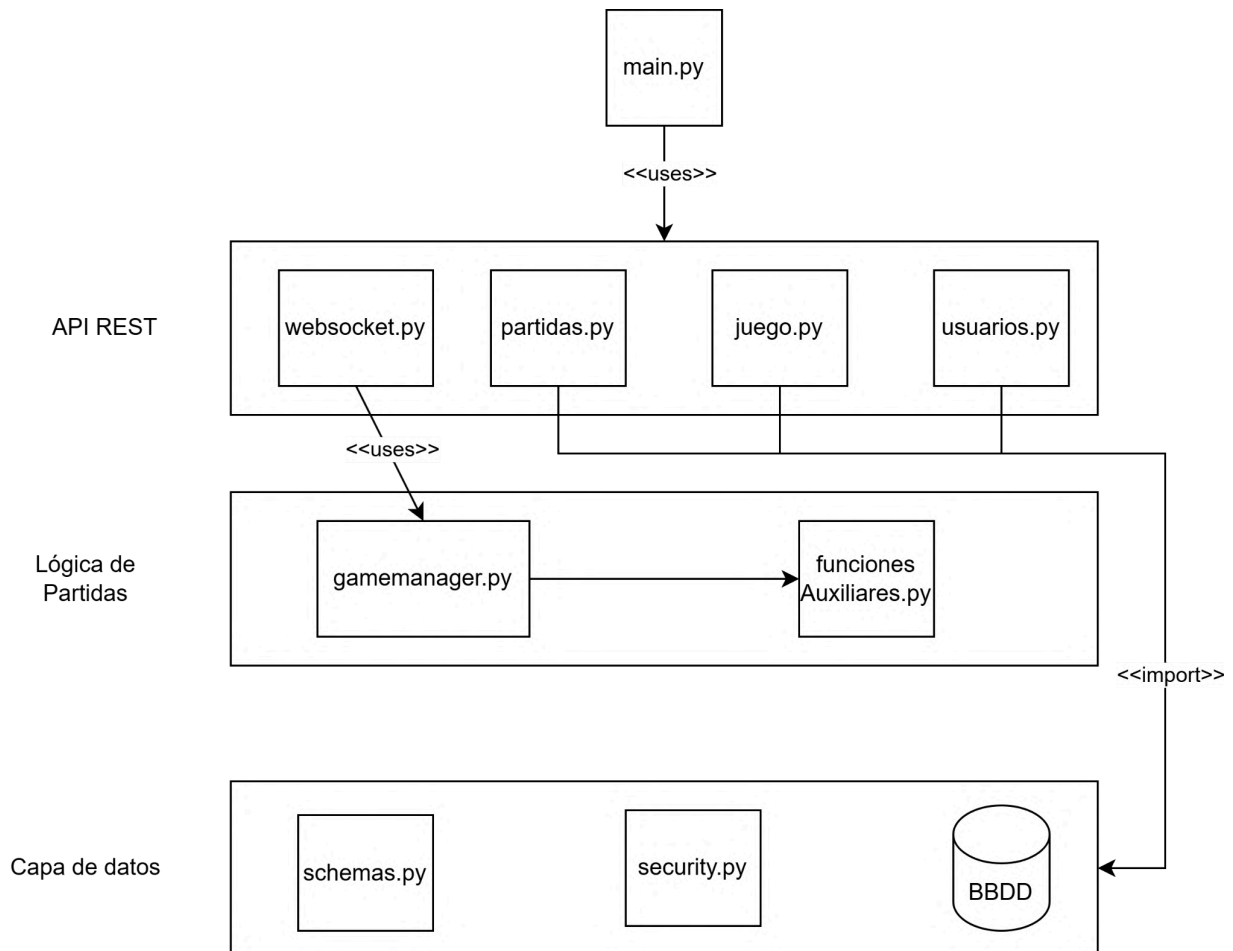


Figura 2: Diagrama de módulos

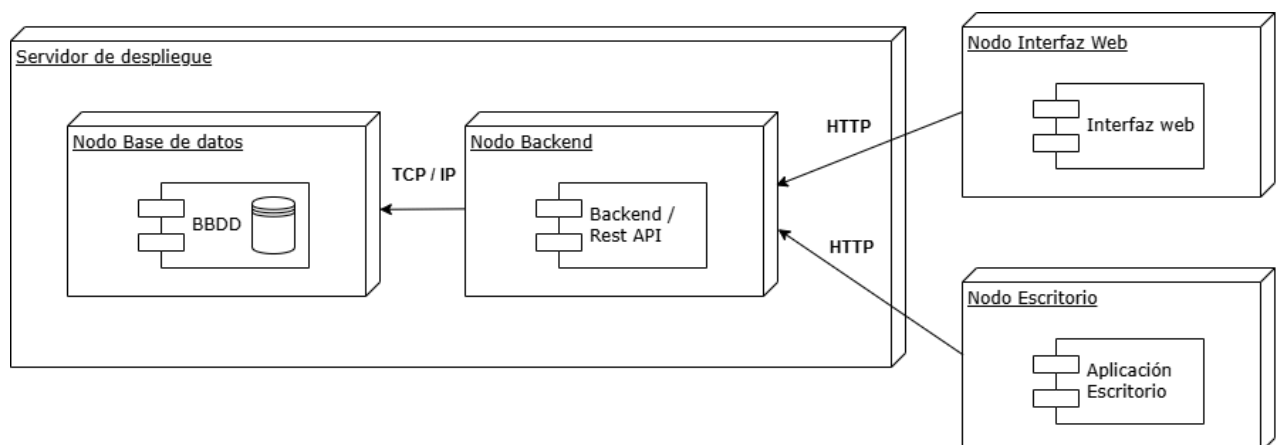


Figura 3: Diagrama de despliegue

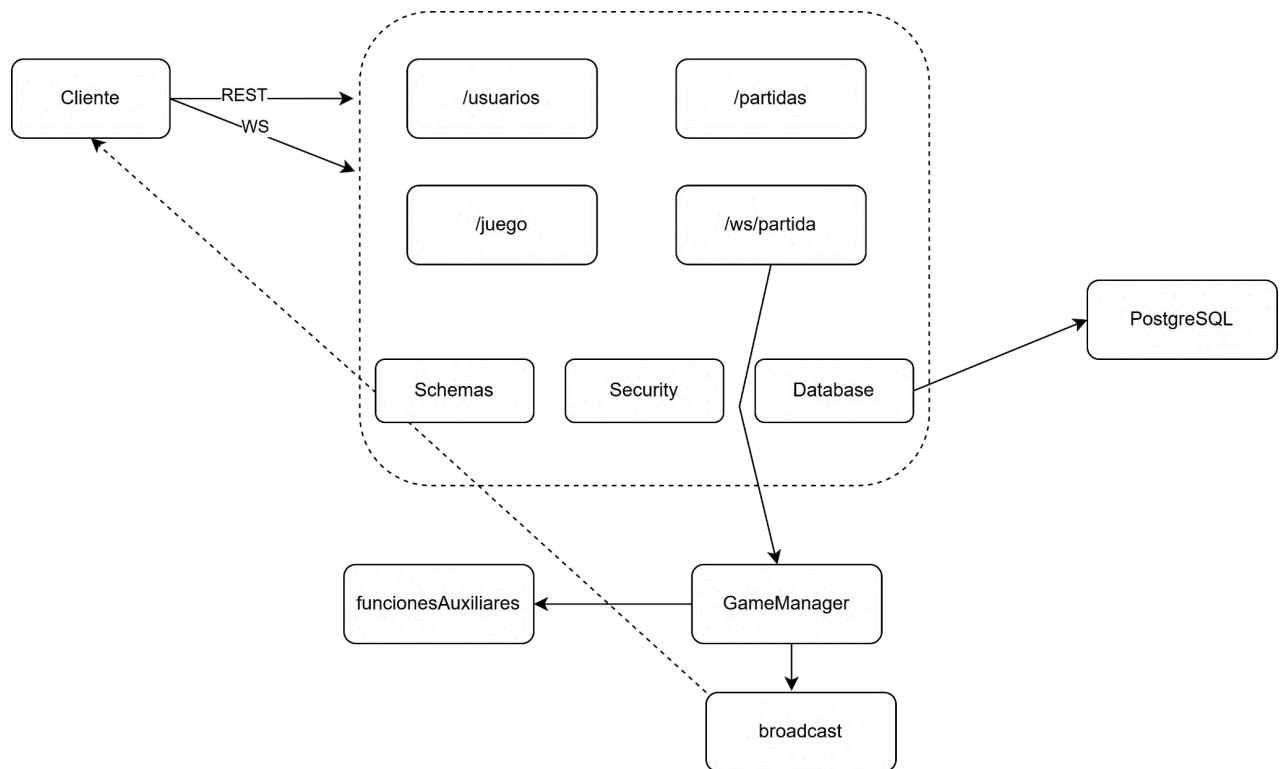


Figura 4: Diagrama de componentes

Diseño gráfico del sistema

Se muestra la línea gráfica y el diseño de personajes, tablero y minijuegos



Figura 5: Logo y personajes del juego

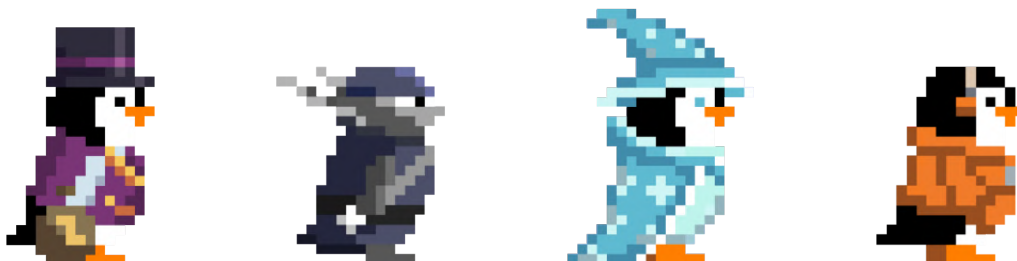
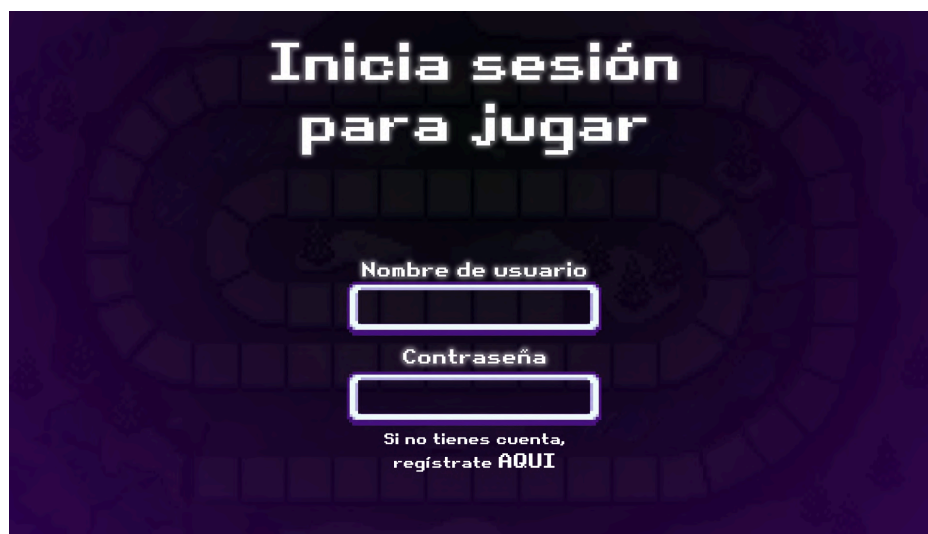


Figura 6: Tablero del juego con las diferentes casillas especiales y vista de los personajes en el tablero



Figura 7: Pantalla previa a la partida - Pantalla principal (DISEÑO PROVISIONAL)



Figuras 8 y 9: Pantallas de registro e Inicio de sesión (DISEÑO PROVISIONAL)

Minijuegos: Todos los diseños pueden variar con los del juego final

Tren: Se muestra el escenario y los diferentes personajes que aparecerán en las ventanas
En el minijuego se moverán múltiples vagones encadenados



Figura 10: Fondo del minijuego - Tren

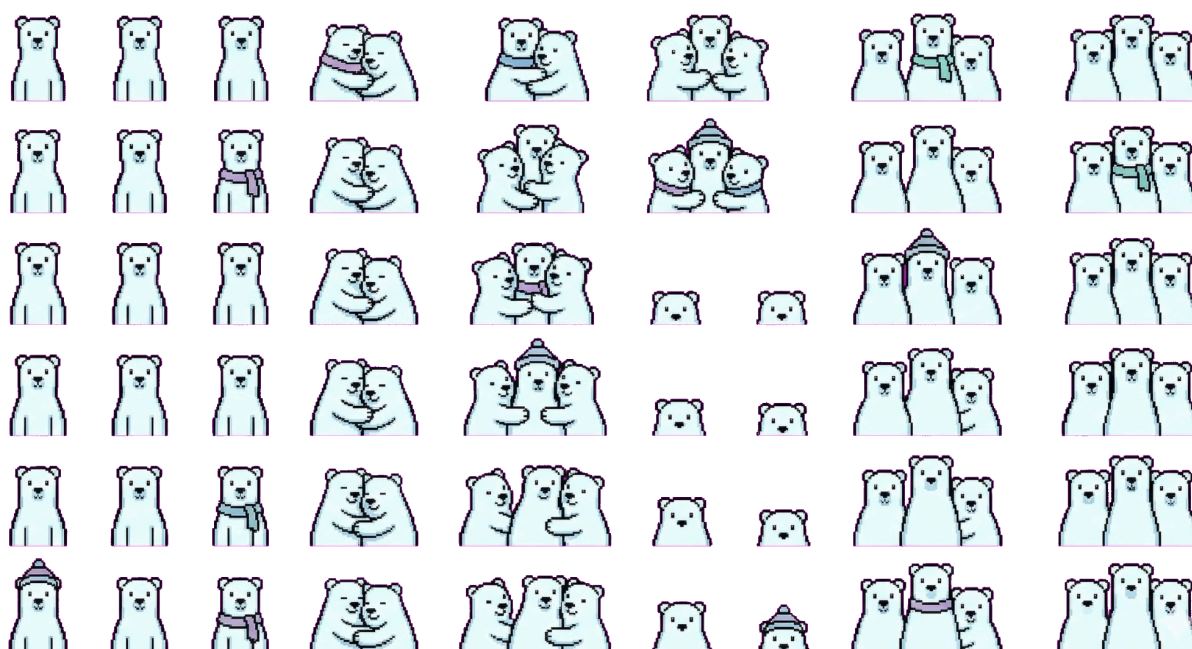


Figura 11: Personajes del minijuego - Tren

Reflejos: Se muestra el escenario del minijuego con el fondo verde/rojo



Figuras 12 y 13: Escenarios del minijuego - Reflejos

Cortar pan: Se muestra el escenario del minijuego



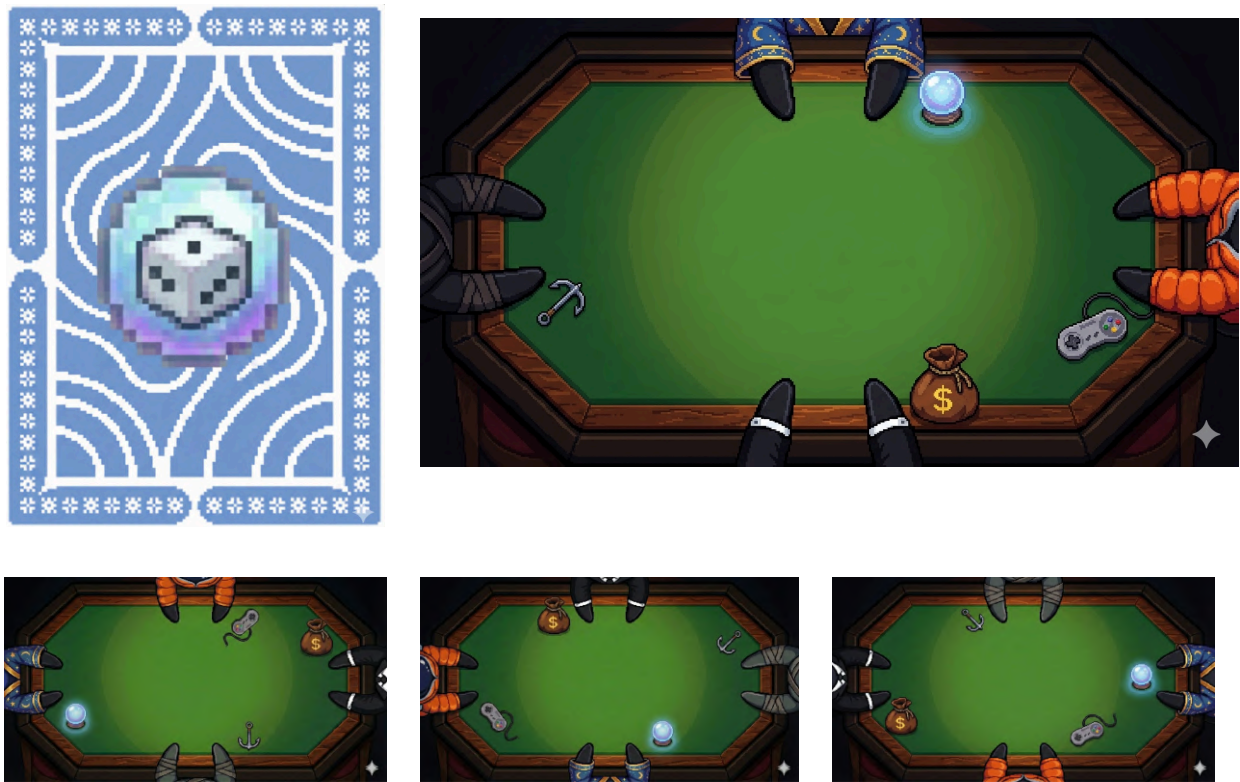
Figura 14: Escenario del minijuego - Cortar Pan

Cronómetro ciego: Se muestra el escenario del minijuego con el cronómetro desvaneciéndose



Figuras 15, 16 y 17: Escenario del minijuego - Cronómetro ciego

Mayor o menor/ mano de poker: Se muestra el diseño de las cartas y el escenario en las diferentes perspectivas, que es igual para ambos minijuegos



Figuras 18, 19, 20, 21 y 22: Carta y escenarios de mesa para juegos de cartas

Dilema del Prisionero: Se muestra el escenario del minijuego



Figura 23: Escenario del minijuego Dilema del Prisionero

Doble o Nada: Se juega en el tablero, no tiene diseño propio

5. Memoria del proyecto

5.1 Inicio del proyecto

Se hace referencia a lo propuesto en el apartado 3.1 y se comenta cómo avanza el proyecto y si se han cumplido los objetivos en cada uno de los apartados a continuación:

Identificación y asignación de recursos

A día de redacción de este documento (15 de marzo), no se ha elegido aún el servidor en el que desplegar, debido a que nos encontramos barajando todavía diferentes opciones, las ya mencionadas.

Para *Evaluación del estado del proyecto (resultado Práctica 4)*, entregable el día 6 de abril, se deberá de haber seleccionado una de las opciones para la prueba del juego.

Gestión de riesgos principales

A día de redacción de este documento (15 de marzo), no se ha producido ningún problema mencionado en el apartado 3.1 en cuanto a riesgos de proyecto. En caso de que se produjeran, se completaría la hoja de cálculo referenciada en ese mismo apartado.

La toma de decisiones centralizada por el coordinador está dando buenos resultados, así como la constante comunicación entre los miembros del equipo. Al ser un juego similar a otro que todos hemos jugado y estar las funcionalidades muy definidas (apartado 4.1, requisitos) no ha habido problemas por esa parte. Por último, ya sabíamos que desconocer alguna tecnología iba a suponer horas dedicadas a aprenderla, por lo que no hemos tenido sorpresas en cuanto a esto.

Plan de formación

Como se ha mencionado en el apartado anterior, ya sabíamos de antemano que se iba a tener que dedicar tiempo al aprendizaje de las nuevas tecnologías y se ha seguido el plan establecido. Se han desarrollado ejemplos prácticos muy sencillos del tipo de los minijuegos o de un tablero con turnos simples que hemos podido trasladar al proyecto de manera que estos han servido de “versión inicial”. Así, se ha partido del tablero de juego en local con dos jugadores para llegar a la implementación de los websockets para turnos conectados al backend, etc.

5.2 Ejecución y control del proyecto

Se hace referencia a lo propuesto en los apartados 3.2 y 3.3 y se comenta cómo avanza el proyecto y si se han cumplido los objetivos en cada uno de los apartados a continuación:

Gestión de repositorios y control de versiones

El flujo de trabajo acordado, sin aprobación estricta de merges, ha funcionado muy bien. Gracias a la comunicación constante y al trabajo en grupo, no hemos experimentado conflictos de código en ningún momento ya que todos éramos conscientes de lo que estaba trabajando cada uno.

Responsables

Eduardo Blasco y Alonso de la Calle han desempeñado su rol adecuadamente, garantizando que los repositorios y la configuración se mantengan operativos para el resto del equipo.

Despliegue e infraestructura

Las tecnologías seleccionadas están funcionando bien, según lo esperado y no nos hemos visto obligados a cambiar ninguna de las decisiones técnicas iniciales. Aunque el sistema no se ha desplegado todavía en un servidor remoto, las pruebas del contenedor de Docker en local (encapsulando el backend y la base de datos) funcionan correctamente.

Convenciones

El equipo ha respetado los comentarios y la estructura de carpetas, lo que ha facilitado la lectura y comprensión del código de otros miembros y la resolución de dudas.

Aseguramiento de la calidad

- **Casos de prueba con Fast Api:** El equipo de backend ha probado satisfactoriamente los endpoints. Además, se ha conseguido con éxito el hito técnico de integrar la API REST con los WebSockets de forma estable.
- **Casos de prueba concretos:** Se están evaluando localmente los escenarios base del tablero y el registro de usuarios. No se han definido los casos de prueba límites ya que aún se está haciendo funcionar una versión inicial del juego.

- **Pruebas cruzadas (Cross-testing):** De momento las pruebas se están limitando al entorno de desarrollo de cada miembro, dejando las pruebas cruzadas completas para cuando las interfaces estén más avanzadas.
- **Sesiones de Playtesting:** Aún no se han realizado sesiones conjuntas de juego. La versión actual se ejecuta en local y el equipo sigue realizando pruebas técnicas individuales antes de poder jugar completamente.

Calendario del proyecto

A nivel de planificación, hemos llegado a la entrega de la iteración intermedia (15 de marzo) demasiado justos de tiempo debido a una mala gestión temporal.

Específicamente, la fase de diseño visual ha requerido mucho más tiempo del estimado; aunque hemos cumplido con tener el diseño base definido, los assets definitivos aún no están terminados (El diseño está, pero no se ha formalizado con las medidas exactas para que ambos equipos de frontend los utilicen y el resultado final sea lo más similar posible).

En el resto de componentes técnicos sí hemos cumplido los plazos. Como propuesta de mejora para las siguientes iteraciones, el equipo ha acordado fijar deadlines internos más estrictos y con varios días de antelación respecto a las fechas de entrega reales, permitiendo así margen de maniobra ante imprevistos. Esto último aplicará a la entrega del 6 de abril.

Monitorización y seguimiento del progreso

La comunicación interna mediante nuestros canales ha sido perfecta.

En cuanto al reparto de tareas, comenzamos a utilizar los Issues de GitHub en la parte de frontend más tarde de lo que deberíamos, pero una vez implementados se han convertido en una herramienta muy útil.

Por último, el registro en el Excel de control horario es un método muy cómodo para que todo el equipo se mantenga actualizado sin perder mucho tiempo, ya que se registran las horas al momento de realizarlas y puedes consultar las horas en una tabla sin tener que realizar cálculos manuales.

5.3 Cierre del proyecto

Comparativa de esfuerzos y horas trabajadas

En la propuesta inicial se estimaron 917 horas de trabajo. A día de redacción de este documento (15 de marzo), se han completado 207,5 horas, lo que supondría alrededor de un 23% de las horas totales.

Consideramos que el ritmo de trabajo es muy bueno pero creemos que la parte de implementación de los minijuegos puede complicarse más que esta primera fase del proyecto por lo que consideramos estar alrededor de un 30/40% del proyecto, sin contar los inconvenientes que puedan surgir, que lo rebajaría a un 25/30%, acorde con el recuento.

Se desglosa a continuación las horas invertidas por cada miembro del equipo y las tareas en las que se han invertido. Se puede consultar con más detalle en la hoja de cálculo referenciada a continuación:

 [Gestion_Horas_PSoft](#)

Desglose por miembro y tarea:

Lorién (Total: 32.5 horas)

- 13 horas en Organización
- 13 horas en Diseño
- 4.5 horas en Prácticas
- 2 horas en Frontend Escritorio

Alonso (Total: 30.5 horas)

- 16.25 horas en API Backend
- 4.5 horas en Prácticas
- 3.5 horas en BD Backend
- 3.25 horas en Propuesta técnica y económica
- 3 horas en Organización

Daniel (Total: 28.5 horas)

- 15.5 horas en Diseño
- 6 horas en Frontend Escritorio
- 4.5 horas en Prácticas
- 2.5 horas en Organización

Eduardo (Total: 28 horas)

- 13.75 horas en API Backend
- 4.5 horas en Prácticas
- 3.5 horas en BD Backend
- 2.5 horas en Organización

- 2 horas en Propuesta técnica y económica
- 1.75 horas en Conexión Front-Back

Aritz (Total: 26.5 horas)

- 11.5 horas en API Backend
- 5.5 horas en BD Backend
- 4.5 horas en Prácticas
- 4 horas en Organización
- 1 hora en Propuesta técnica y económica

Alejandro (Total: 23.75 horas)

- 11 horas en Frontend Escritorio
- 5.75 horas en Propuesta técnica y económica
- 4.5 horas en Prácticas
- 2.5 horas en Organización

Juanma (Total: 20.5 horas)

- 4.5 horas en Prácticas
- 3.5 horas en Frontend Web
- 3.5 horas en Organización
- 3.25 horas en Propuesta técnica y económica
- 2.5 horas en Frontend (General)
- 1.75 horas en Conexión Front-Back
- 1.5 horas en Diseño

David (Total: 17.25 horas)

- 7.75 horas en Frontend Web
- 4.5 horas en Prácticas
- 2.5 horas en Frontend (General)
- 1.5 horas en Organización
- 1 hora en Propuesta técnica y económica

Horas totales: 207,5

Análisis de riesgos y problemas imprevistos

En el apartado 5.1, se comenta que los riesgos previstos no se han producido y se explica más en detalle. Como riesgo imprevisto, cabe destacar el tiempo que se ha de invertir en la “formalización” de los assets. Al no ser solo el diseño y tener que pasarlo a un formato estándar, no hemos podido realizar el prototipo inicial de la aplicación con los gráficos finales, pero lo consideramos un problema menor ya que se incluirá en la versión final.

Lecciones aprendidas

A lo largo de esta primera mitad del proyecto, el equipo ha extraído conclusiones muy valiosas para aplicar en el futuro:

- **Creación de assets:** La formalización y adaptación de los gráficos para que sean utilizables en el frontend consume mucho más tiempo del estimado inicialmente.
- **Documentación:** La redacción de memorias y documentos técnicos rigurosos también ocupa más tiempo del esperado.
- **Visión del proyecto:** El planteamiento general inicial ha sido muy acertado. El hecho de que todo el equipo conociera y comprendiera perfectamente la idea central del juego desde el primer día ha evitado malentendidos durante el desarrollo.
- **Github:** Aprender a utilizar GitHub correctamente es complicado, pero dominarlo es muy útil e imprescindible para la coordinación del equipo. *En trabajos anteriores de la carrera hubo muchos problemas por el uso de github, problemas de merges, etc.*

Anexo I - Glosario

API REST

Interfaz de comunicación entre sistemas basada en el protocolo HTTP, que permite el intercambio de información mediante operaciones estándar como GET, POST, PUT y DELETE.

Backend

Parte del sistema encargada de la lógica de negocio, gestión de datos y comunicación con los clientes, ejecutada en el servidor.

Contenerización

Técnica que permite empaquetar una aplicación junto con todas sus dependencias en contenedores, facilitando su despliegue y portabilidad.

Docker

Plataforma de software utilizada para la creación, despliegue y ejecución de aplicaciones mediante contenedores.

Frontend

Parte del sistema encargada de la interfaz de usuario y de la interacción directa con este.

MVP (Producto Mínimo Viable)

Versión inicial de un producto que incluye únicamente las funcionalidades esenciales para validar su funcionamiento.

Multiplataforma

Capacidad de una aplicación para ejecutarse en diferentes entornos o sistemas operativos, como web y escritorio.

Tiempo real

Característica de un sistema que permite la sincronización inmediata de acciones y datos entre múltiples usuarios conectados.

PaaS (Platform as a Service)

Modelo de servicio en la nube que proporciona una plataforma completa para el desarrollo, despliegue y gestión de aplicaciones, sin necesidad de administrar la infraestructura subyacente.

WebSocket

Protocolo de comunicación bidireccional y persistente sobre TCP que permite el intercambio de mensajes en tiempo real entre cliente y servidor sin necesidad de realizar nuevas peticiones HTTP.

VPS (Virtual Private Server)

Servidor privado virtual que simula un servidor dedicado dentro de un entorno de alojamiento compartido, ofreciendo recursos garantizados y acceso root al sistema operativo.

Broadcast

Operación mediante la cual el servidor envía un mensaje simultáneamente a todos los clientes conectados a una sesión, utilizada para sincronizar el estado de la partida entre jugadores.

Flutter

Framework de desarrollo de interfaces de usuario creado por Google que permite compilar aplicaciones nativas para escritorio, móvil y web desde una única base de código en Dart.

FastAPI

Framework web de Python orientado a la creación de APIs, que genera documentación interactiva automáticamente y ofrece alto rendimiento mediante programación asíncrona.

Cross-testing

Práctica de calidad en la que los desarrolladores de un componente validan el funcionamiento de otro componente distinto al que han desarrollado, reduciendo el sesgo de la revisión.